Oefening 15 - MVCVoertuig - Voorbeeld PE: stappenplan

Het volledige project staat in deze map. Leer het in de volgorde hieronder: dat is ook de volgorde waarin je het in de PE bouwt.

Start in Visual Studio: *ASP.NET Core Web App (Model-View-Controller)*, framework **.NET 10**, Authentication: *None*.

---

1. NuGet-packages

Klik rechts op het project → *Manage NuGet Packages* → Browse. Installeer:

- `Microsoft.EntityFrameworkCore.SqlServer` voor de verbinding met SQL Server
- `Microsoft.EntityFrameworkCore.Tools` voor `Add-Migration` en `Update-Database`

Neem dezelfde hoofdversie als .NET (10.x).

2. DbContext → `Data/VoertuigDbContext.cs`

```
public class VoertuigDbContext : DbContext
{
    public VoertuigDbContext(DbContextOptions<VoertuigDbContext> options) : base(options) { }

    public DbSet<Voertuig> Voertuigen { get; set; } = null!;   // stap 9: tabel Voertuigen
}
```

Om te onthouden: je erft over van `DbContext`. De constructor krijgt `DbContextOptions<...>` binnen en geeft die door met `: base(options)`. De naam van de `DbSet` wordt de **tabelnaam**.

3. appsettings.json

```
"ConnectionStrings": {
  "VoertuigConnection": "Server=
(localdb)\\MSSQLLocalDB;Database=VoertuigDb;Trusted_Connection=True;MultipleActiveResultSets=true"
}
```

⚠️ Schrijf **twee** backslashes in `(localdb)\\MSSQLLocalDB`, want het is JSON. Zet een komma na het vorige blok (`"AllowedHosts": "*",`).

4. Program.cs: de DbContext opstarten

Zet dit **vóór** `builder.Build()`:

```
var connectionString = builder.Configuration.GetConnectionString("VoertuigConnection");
builder.Services.AddDbContext<VoertuigDbContext>(
    options => options.UseSqlServer(connectionString));
```

Usings: `using Microsoft.EntityFrameworkCore;` en `using MVCVoertuig.Data;`

5. Model → `Models/Voertuig.cs`

Eigenschap	Annotaties
`int VoertuigId`	geen nodig: `<Klasse>Id` wordt automatisch de primary key
`string Merk`	`[Required]` `[StringLength(100)]`
`string MerkType`	`[Required]` `[StringLength(100)]`
`int Bouwjaar`	`[Required]` `[Bouwjaar]` (dat is de custom validatie)
`int Km`	`[Required]` `[Range(0, 5000000)]`
`decimal VerkoopPrijs`	`[Required]` `[Column(TypeName = "decimal(8,2)")]`
`bool AanwezigInShowroom`	niets (niet verplicht)

Usings: `System.ComponentModel.DataAnnotations` (voor Required, StringLength en Range) en `System.ComponentModel.DataAnnotations.Schema` (voor **Column**).

6. Custom validatie → `Validation/BouwjaarAttribute.cs`

```
public class BouwjaarAttribute : ValidationAttribute
{
    protected override ValidationResult? IsValid(object? value, ValidationContext validationContext)
    {
        int bouwjaar = Convert.ToInt32(value);
        if (bouwjaar < 1900 || bouwjaar > DateTime.Now.Year)
            return new ValidationResult($"Bouwjaar moet tussen 1900 en {DateTime.Now.Year} liggen");
        return ValidationResult.Success;
    }
}
```

- Je erft over van `ValidationAttribute` en override `IsValid`.
- De klassenaam eindigt op `Attribute`. In het model schrijf je `[Bouwjaar]`, zonder "Attribute".
- Waarom kan `[Range(1900, 2026)]` niet? Het huidige jaar moet **dynamisch** zijn, en een attribuut aanvaardt alleen constanten. Daarom bestaat deze custom klasse.
- Deze validatie draait **op de server**. Je ziet de fout pas na "Opslaan", omdat `ModelState.IsValid` dan false is.

7 + 8. VoertuigController: Index en Create

Klik rechts op Controllers → Add → Controller → *MVC Controller – Empty* → `VoertuigController`.

```
private readonly VoertuigDbContext _context;
public VoertuigController(VoertuigDbContext context) { _context = context; }   // DI

public IActionResult Index() => View(_context.Voertuigen.ToList());

[HttpGet]
public IActionResult Create() => View();

[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Create(Voertuig voertuig)
{
    if (ModelState.IsValid)
    {
        _context.Voertuigen.Add(voertuig);
        _context.SaveChanges();
        return RedirectToAction(nameof(Index));
    }
    return View(voertuig);   // toont de foutmeldingen
}
```

Views: klik rechts in de actie → *Add View* → *Razor View*. Kies template **List** (Index) of **Create** en als model **Voertuig**. Visual Studio genereert dan het formulier.

9. Tabel Voertuigen aanmaken met migrations

Open *Tools* → *NuGet Package Manager* → *Package Manager Console*:

```
Add-Migration initial
Update-Database
```

Krijg je een fout? Controleer of het project kan bouwen, of de naam van de connection string in `appsettings.json` en `Program.cs` **identiek** is, en of Tools geïnstalleerd is.

10. Default data → `Data/SeedData.cs` en Program.cs

```
public static void EnsurePopulated(WebApplication app)
{
    using (var scope = app.Services.CreateScope())
    {
        var context = scope.ServiceProvider.GetRequiredService<VoertuigDbContext>();
        if (!context.Voertuigen.Any())
        {
            context.Voertuigen.AddRange(new Voertuig { Merk = "AUDI", ... }, ...);
            context.SaveChanges();
        }
    }
}
```

In Program.cs zet je dit **na** `builder.Build()` en **vóór** `app.Run()`:

```
SeedData.EnsurePopulated(app);
```

`CreateScope()` is nodig omdat de DbContext *scoped* is. `if (!...Any())` zorgt dat de data niet bij elke start opnieuw toegevoegd wordt.

11. Styling → `Views/Shared/_Layout.cshtml`

Vervang de `<header>`, de container en de `<footer>` door de code uit de opgave. Laat de `<head>` en de `<script>`-tags staan, want anders werken Bootstrap en de validatie niet meer.

12. Partial View → `Views/Shared/_VoertuigCard.cshtml`

- Bovenaan zet je `@model Voertuig`, daaronder de card-HTML uit de opgave.
- De naam begint volgens de afspraak met `_`.
- Gebruik in `Voertuig/Index.cshtml`:

```
@model IEnumerable<Voertuig>
@foreach (var voertuig in Model)
{
    <partial name="_VoertuigCard" model="voertuig" />
}
```

13. TagHelper → `TagHelpers/CarInfoTagHelper.cs`

```
public class CarInfoTagHelper : TagHelper
{
    public int Aantal { get; set; }

    public override void Process(TagHelperContext context, TagHelperOutput output)
    {
        output.TagName = "div";
        output.TagMode = TagMode.StartTagAndEndTag;
        output.Attributes.SetAttribute("class", "btn btn-outline-primary m-3");
        output.Content.SetHtmlContent($"Aantal voertuigen <span class=\"badge bg-primary\">{Aantal}</span>");
    }
}
```

- Gebruik: `<car-info aantal="@Model.Count()"></car-info>`
- `CarInfoTagHelper` wordt de tag `<car-info>`, en de property `Aantal` wordt het attribuut `aantal`.
- ⚠️ Vergeet niet `@addTagHelper *, MVCVoertuig` in `Views/_ViewImports.cshtml`. Daar staat de **projectnaam** (de assembly). Zonder die regel gebeurt er niets en zie je gewoon `<car-info>` in de HTML.
- Usings: `Microsoft.AspNetCore.Razor.TagHelpers`

14. ViewComponent → `ViewComponents/HeaderMenuViewComponent.cs`

```
public class HeaderMenuViewComponent : ViewComponent
{
    private readonly VoertuigDbContext _context;
    public HeaderMenuViewComponent(VoertuigDbContext context) { _context = context; }

    public IViewComponentResult Invoke()
    {
        var merken = _context.Voertuigen.Select(v => v.Merk).Distinct().OrderBy(m => m).ToList();
        return View(merken);
    }
}
```

- De view staat **verplicht** in `Views/Shared/Components/HeaderMenu/Default.cshtml`. Je gebruikt de naam zonder "ViewComponent", en het bestand heet *Default*.

```
@model IEnumerable<string>
@foreach (var merk in Model)
{
    <a class="btn btn-info m-1" asp-controller="Voertuig" asp-action="Index" asp-route-merk="@merk">@merk</a>
}
```

- In `_Layout.cshtml`, in de `<nav>`: `@await Component.InvokeAsync("HeaderMenu")`
- `Distinct()` geeft de **unieke** merken. Zo staat AUDI er maar één keer, ook al zijn er 2 Audi's.

- Extra: `Index(string? merk)` in de controller filtert op merk wanneer je op een merk klikt.

TagHelper vs ViewComponent vs Partial View

	Wat	Heeft eigen C#-logica / DB?
Partial View	stukje HTML dat je hergebruikt en dat een model krijgt	nee, krijgt het model van de view
TagHelper	eigen HTML-tag, verwerkt door C# (`Process`)	logica wel, maar meestal geen DB
ViewComponent	mini-controller met een eigen view	ja: DI van de DbContext, haalt zelf data op

Checklist op het einde

- [ ] Bouwt het project? (Ctrl+Shift+B)
- [ ] Is `Add-Migration` + `Update-Database` gedaan?
- [ ] Staat de default data er na het opstarten?
- [ ] Geeft Create met bouwjaar 1850 of 2099 een foutmelding?
- [ ] Geeft Km = 6000000 een foutmelding?
- [ ] Toont het menu Home, Voertuigen en alle unieke merken?
- [ ] Is `<car-info>` een knop met badge, en niet gewoon tekst?